



LAB REPORT- 04

Department of Computer Science & Engineering (CSE)

Faculty of Science and Information Technology (FSIT)

Summer 2026

Course Title : Computer Graphics Lab
Course Code : CSE412
Course Credit : 1.5
Date of Submission : July 14, 2026
Topic Covered : Implementation of Midpoint Circle
Drawing Algorithm Using OpenGL

Submitted By

Shafin Ahmed

232-15-184

65_M1

Department of Computer Science & Engineering

Submitted To

Husne Mubarak

Lecturer

Department of Computer Science & Engineering

Faculty of FSIT

Comment By Course Teacher:

--

Lab Report 04

Title: Implementation of Midpoint Circle Drawing Algorithm Using OpenGL

1. Introduction

Computer Graphics is a branch of computer science that deals with creating, displaying, and manipulating graphical objects using computer programs. One of the most fundamental operations in computer graphics is drawing geometric primitives such as lines, circles, and polygons. Since a computer screen is made up of discrete pixels rather than continuous mathematical coordinates, efficient algorithms are required to determine which pixels should be illuminated in order to represent a shape accurately.

The **Midpoint Circle Drawing Algorithm** is one of the most efficient raster graphics algorithms for drawing circles. Instead of using the circle equation and expensive floating-point calculations, it uses a decision parameter to determine the next pixel position. This significantly improves execution speed while maintaining the symmetry and accuracy of the circle. The algorithm computes the points of only one octant and then generates the remaining seven octants using the symmetry property of a circle.

In this experiment, a **single Midpoint Circle Drawing Algorithm** was implemented using **OpenGL** in the Code::Blocks IDE. The program accepts the radius and the center coordinates (x_c , y_c) from the user, automatically calculates the required points, and displays the circle on the screen. The coordinate system was modified to **0–100** for both the X-axis and Y-axis, the OpenGL window size was set to **700 × 700**, and the drawing color was specified using the **RGB (0–255)** color format.

2. Environment / Platform

The following software and libraries were used to perform this experiment.

Component	Description
Programming Language	C++
IDE	Code::Blocks
Graphics Library	OpenGL (GLUT)
Compiler	GNU GCC Compiler

Operating System	Windows 10 / Windows 11
Window Size	700 × 700 pixels
Coordinate System	X-axis: 0–100, Y-axis: 0–100
Color Format	RGB (0–255)

Before implementing the algorithm, the OpenGL and GLUT libraries were properly configured in the Code::Blocks project. The projection system was initialized using **glOrtho()** to create a two-dimensional coordinate system ranging from **0 to 100** along both axes. The background color was configured using **glClearColor()**, and the graphics window was created using the GLUT initialization functions.

3. Problem Statement

Experiment

Design and implement a **single Midpoint Circle Drawing Algorithm** using OpenGL.

The program should:

- Accept the radius from the user.
- Accept the center coordinates (**xc, yc**).
- Draw the circle using the Midpoint Circle Drawing Algorithm.
- Use the eight-way symmetry property of the circle.
- Change both the X-axis and Y-axis to **0–100**.
- Convert the OpenGL color from the default normalized format to the **RGB (0–255)** format.
- Set the OpenGL window size to **700 × 700** pixels.
- Display the generated circle accurately on the graphics window.

4. Objectives

The objectives of this experiment are:

- To understand the working principle of the Midpoint Circle Drawing Algorithm.
- To implement circle generation using integer arithmetic.
- To learn how the decision parameter selects the next pixel.
- To understand the eight-way symmetry property of a circle.
- To draw circles centered at any arbitrary coordinate.
- To practice OpenGL programming using GLUT.
- To modify the OpenGL coordinate system according to laboratory requirements.
- To apply RGB (0–255) color representation in OpenGL.

5. Algorithm (Midpoint Circle Drawing Algorithm)

1. Read the radius **r** from the user.
2. Read the center coordinates (**xc, yc**).
3. Initialize:
 - $x = 0$
 - $y = r$
 - Decision Parameter $p = 1 - r$
4. Plot the eight symmetric points of the circle.
5. Repeat until $x > y$:
 - If $p < 0$
 - Increase x
 - Update the decision parameter.
 - Otherwise
 - Increase x
 - Decrease y
 - Update the decision parameter accordingly.
6. Continue plotting all eight symmetric points.
7. Display the completed circle.

6. Code Strategy

The implementation follows the standard Midpoint Circle Drawing Algorithm discussed during the laboratory session. The program first receives the radius and the center coordinates from the user. It initializes the decision parameter and begins plotting points from the first octant. For every calculated point, the remaining seven symmetric points are generated automatically using the circle's symmetry property.

The OpenGL window is configured with a **700 × 700** display size and a **0–100** coordinate system. The circle color is defined using the RGB **0–255** format by dividing each color component by **255** before passing it to `glColor3f()`. This implementation avoids floating-point calculations during the circle generation process, making it efficient and suitable for raster graphics applications.

7. Source Code

```
// Midpoint Circle Drawing Algorithm
// Shafin Ahmed
// ID: 232-15-184
// Section: 65-M1
// Lab Report 04
// Circle with Center (xc, yc)

#include <GL/gl.h>
#include <GL/glut.h>
#include <stdio.h>
```

```

int r;
int xc, yc;

void init()
{
    // Set white background
    glClearColor(1.0, 1.0, 1.0, 1.0);

    // Projection setup
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Coordinate system (0 - 100)
    glOrtho(0, 100, 0, 100, -1, 1);

    // Increase point size for better visibility
    glPointSize(3.5);
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);

    // Royal Blue (RGB 0-255)
    float Red = 65;
    float Green = 105;
    float Blue = 225;

    glColor3f(Red/255.0, Green/255.0, Blue/255.0);

    glBegin(GL_POINTS);

    int x = 0;
    int y = r;

    // Initial decision parameter
    int p = 1 - r;

    while (x <= y)
    {
        // Plot all eight symmetric points

        glVertex2i(xc + x, yc + y);
        glVertex2i(xc + y, yc + x);

        glVertex2i(xc + y, yc - x);
        glVertex2i(xc + x, yc - y);

        glVertex2i(xc - x, yc - y);
        glVertex2i(xc - y, yc - x);

        glVertex2i(xc - y, yc + x);
        glVertex2i(xc - x, yc + y);
    }
}

```

```

        // Update decision parameter

        if (p < 0)
        {
            x = x + 1;
            p = p + (2 * x) + 1;
        }
        else
        {
            x = x + 1;
            y = y - 1;
            p = p + (2 * x) + 1 - (2 * y);
        }
    }

    glEnd();

    glFlush();
}

int main(int argc, char **argv)
{
    printf("Enter Radius : ");
    scanf("%d", &r);

    printf("Enter Center (X Axis) : ");
    scanf("%d", &xc);

    printf("Enter Center (Y Axis) : ");
    scanf("%d", &yc);

    glutInit(&argc, argv);

    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);

    // Window size
    glutInitWindowSize(700, 700);

    glutInitWindowPosition(100, 80);

    glutCreateWindow("Midpoint Circle Drawing Algorithm");

    init();

    glutDisplayFunc(display);

    glutMainLoop();

    return 0;
}

```

8. Code Description

The program is implemented using the standard **Midpoint Circle Drawing Algorithm** discussed during the laboratory session. It follows the same algorithmic steps while modifying the coordinate system and color format according to the laboratory requirements.

The implementation consists of the following major parts.

Header Files

The program includes the required OpenGL and GLUT header files.

- **GL/gl.h** provides the basic OpenGL drawing functions.
- **GL/glut.h** is used for window creation and event handling.
- **stdio.h** is used for taking user input.

These libraries are sufficient for implementing the Midpoint Circle Drawing Algorithm.

Global Variables

Three global variables are declared.

- **r** stores the radius of the circle.
- **xc** stores the x-coordinate of the center.
- **yc** stores the y-coordinate of the center.

These values are entered by the user during program execution.

Initialization Function

The `init()` function initializes the graphics environment.

The following tasks are performed inside this function:

- Sets the background color.
- Initializes the projection matrix.
- Creates a two-dimensional coordinate system.
- Changes the X-axis and Y-axis from **0 to 100**.
- Prepares the OpenGL drawing environment before rendering.

Display Function

The `display()` function contains the complete implementation of the Midpoint Circle Drawing Algorithm.

Inside this function:

- The graphics window is cleared.
- A drawing color is selected using the RGB (0–255) format.

- The initial values are assigned:
 - $x = 0$
 - $y = r$
 - $p = 1 - r$
- A loop executes until **x becomes greater than y**.
- During each iteration, the algorithm updates the decision parameter.
- Eight symmetric points of the circle are plotted for every calculated point.
- Finally, all points are displayed on the screen using `glFlush()`.

Midpoint Circle Logic

The algorithm begins from the top-most point of the circle.

Initially,

- $x = 0$
- $y = r$
- Decision Parameter = $1 - r$

For every iteration,

- If the decision parameter is negative, only x is increased.

Otherwise,

- x is increased.
- y is decreased.
- The decision parameter is updated.

This process continues until **x becomes greater than y**, ensuring that all pixels of the first octant are calculated correctly.

Eight-Way Symmetry

Instead of calculating every point on the circle individually, the algorithm calculates only one octant.

Using symmetry, the remaining seven points are generated automatically.

For every calculated point (x, y), the following eight points are plotted.

- $(x_c + x, y_c + y)$
- $(x_c + y, y_c + x)$
- $(x_c + y, y_c - x)$
- $(x_c + x, y_c - y)$
- $(x_c - x, y_c - y)$
- $(x_c - y, y_c - x)$
- $(x_c - y, y_c + x)$

- $(xc - x, yc + y)$

This significantly reduces computation while maintaining a perfectly symmetric circle.

Color Selection

The circle color is specified using the RGB (0–255) color format.

Instead of writing normalized values directly, each RGB component is divided by **255** before passing it to `glColor3f()`.

Example:

- Red = 30
- Green = 144
- Blue = 255

This method follows the laboratory requirement and makes color selection easier.

Window Configuration

The graphics window is configured with the following settings.

Property	Value
Window Size	700 × 700 pixels
Coordinate System	X-axis: 0–100, Y-axis: 0–100
Projection	Orthographic (2D)
Circle Center	User Defined (xc, yc)
Radius	User Defined

Input

The program accepts the following values from the user.

- Radius of the circle
- X-coordinate of the center
- Y-coordinate of the center

Example:

Enter Radius : 25

Enter Center (X Axis) : 50

Enter Center (Y Axis) : 50

Working Procedure

The overall execution of the program can be summarized as follows.

1. Read the radius.
2. Read the center coordinates.
3. Initialize OpenGL.
4. Create the graphics window.
5. Set the coordinate system.
6. Initialize the decision parameter.
7. Calculate one octant of the circle.
8. Generate the remaining seven octants using symmetry.
9. Display the completed circle.
10. Keep the graphics window active until it is closed by the user.

Summary of the Program

The program successfully implements the **Midpoint Circle Drawing Algorithm** using OpenGL. It accepts a user-defined radius and center coordinates, generates circle points efficiently using integer arithmetic, applies eight-way symmetry to reduce computation, and displays the circle within a **700 × 700** graphics window using a **0–100** coordinate system. The RGB color is specified using the **0–255** format, satisfying all the laboratory requirements.

9. Output

After successful compilation and execution, the program prompts the user to enter the radius and the center coordinates of the circle. Based on these inputs, the Midpoint Circle Drawing Algorithm calculates the required pixel positions and displays a circle centered at the specified point.

For this experiment, the following sample input was used:

```
D:\glutecode\eightstars\Two | x + v
Enter Radius : 25
Enter Center (X Axis) : 50
Enter Center (Y Axis) : 50
```

The program successfully generated a circle centered at **(50, 50)** with a radius of **25**. The circle was displayed inside a **700 × 700** OpenGL window using a **0–100 coordinate system**. The circle was drawn using the RGB (0–255) color format on a white background, making the output clear and visually attractive.

Output Screenshot

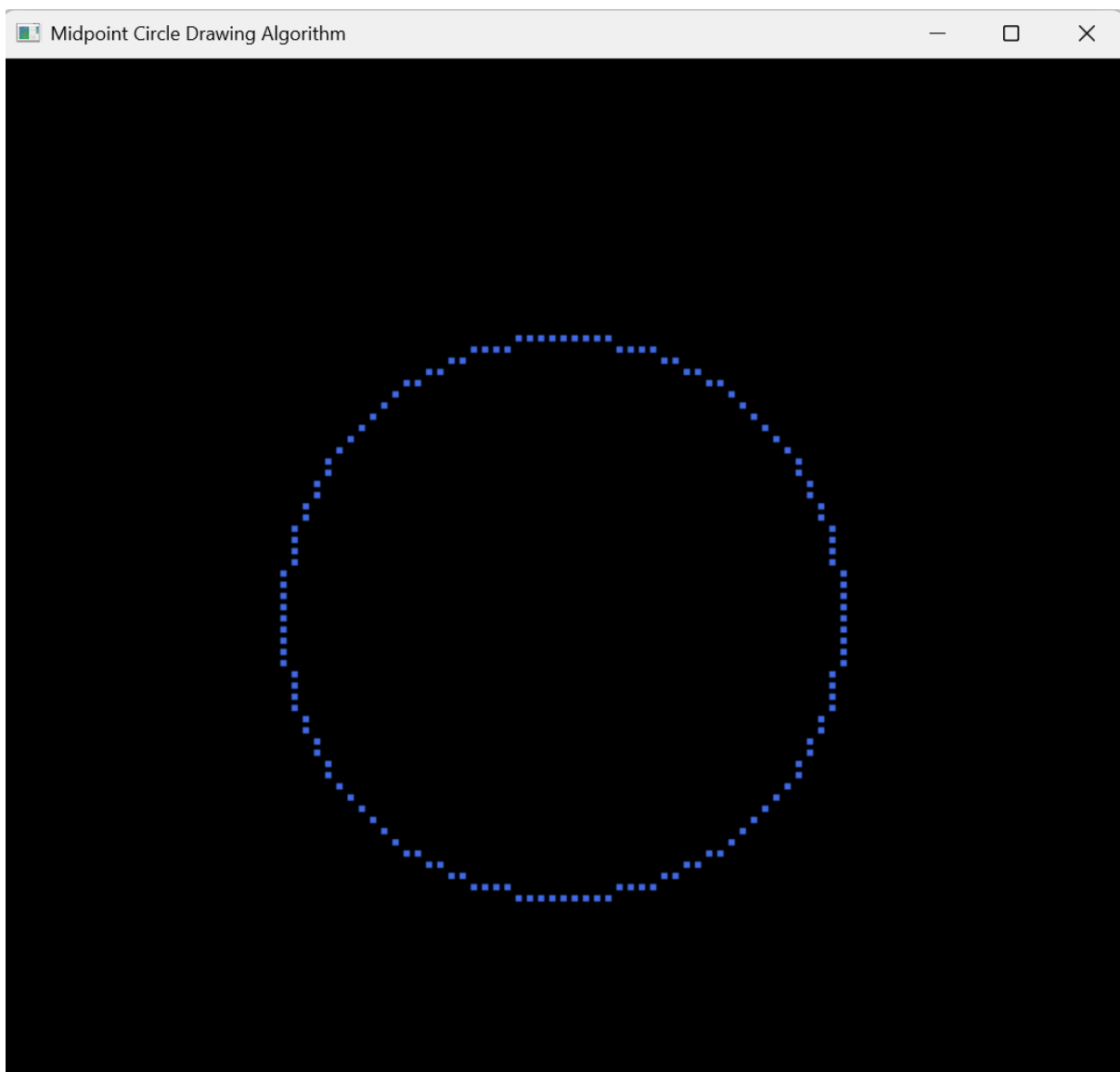


Figure 1: Output of Midpoint Circle Drawing Algorithm

10. Graph (DESMOS)

To verify the correctness of the generated circle, the output was also plotted using **DESMOS Graphing Calculator**. The center point and radius used in the OpenGL program were applied in the graph to visualize the circle mathematically.

The following equation was used:

$$(x-50)^2+(y-50)^2=25^2$$

This graph confirms that the generated circle is centered at **(50,50)** with a radius of **25**, matching the OpenGL output.

DESMOS Equation

$$(x-50)^2+(y-50)^2=25^2$$

Center Point: (50,50)

Radius: 25

Graph

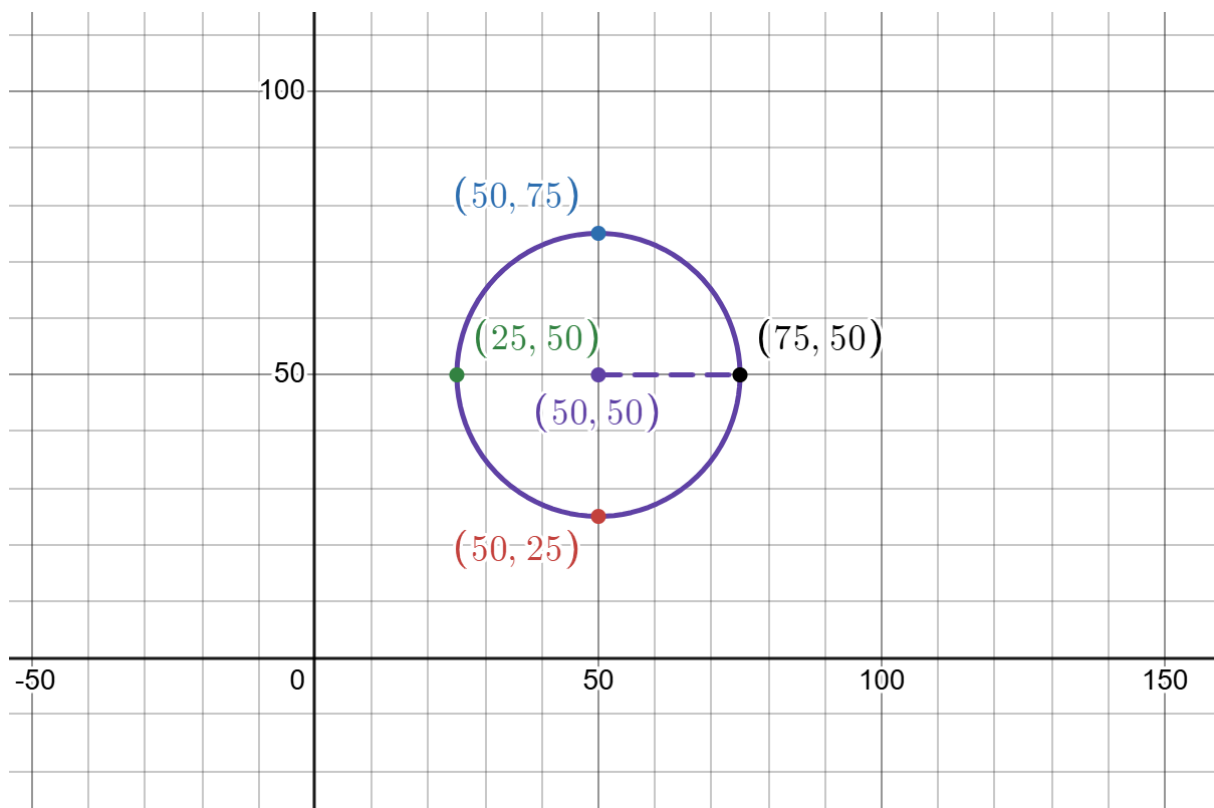


Figure 2: DESMOS Graph of the Circle

11. Discussion

The objective of this experiment was to implement the **Midpoint Circle Drawing Algorithm** using OpenGL. The algorithm successfully generated a circle by calculating only one octant and producing the remaining seven octants using symmetry. This greatly reduced the number of calculations required while maintaining the accuracy of the circle.

Unlike the direct circle equation method, the Midpoint Circle Algorithm uses only integer arithmetic and a decision parameter to determine the next pixel position. As a result, the execution becomes faster and more efficient, making it suitable for raster graphics systems.

The coordinate system was successfully modified to **0–100**, and the OpenGL window size was changed to **700 × 700** according to the laboratory requirements. The RGB (0–255) color format was also implemented, making color selection more convenient than using normalized values.

During implementation, careful attention was required while plotting the eight symmetric points. Incorrect placement of any one point resulted in an incomplete or distorted circle. After correcting the plotting sequence and verifying the decision parameter updates, the algorithm generated the expected output successfully.

Overall, this experiment improved my understanding of raster graphics, symmetry, decision parameters, and efficient circle generation algorithms in computer graphics.

12. Discussion Questions

1. Why does the Midpoint Circle Algorithm compute only one octant?

Because a circle is symmetric about the x-axis, y-axis, and diagonals. After calculating one octant, the remaining seven octants are obtained by symmetry, reducing computation time.

2. What is the purpose of the decision parameter?

The decision parameter determines which pixel is closer to the actual circle boundary. It helps decide whether only the x-coordinate should change or both x and y should be updated.

3. Why does the algorithm stop when $x > y$?

When **x becomes greater than y**, all points in the first octant have already been calculated. Continuing beyond this point would generate duplicate points due to symmetry.

4. Why is the Midpoint Circle Algorithm more efficient than using the circle equation directly?

The circle equation requires floating-point operations and square-root calculations, which are computationally expensive. The Midpoint Circle Algorithm uses only integer addition and subtraction, making it significantly faster.

5. How are the remaining seven points obtained from a single calculated point?

After calculating one point (x, y) , the algorithm uses eight-way symmetry to generate the following points:

- $(x_c + x, y_c + y)$
- $(x_c + y, y_c + x)$
- $(x_c + y, y_c - x)$
- $(x_c + x, y_c - y)$
- $(x_c - x, y_c - y)$
- $(x_c - y, y_c - x)$
- $(x_c - y, y_c + x)$
- $(x_c - x, y_c + y)$

Thus, only one calculation produces eight pixels on the circle.

6. How would you modify the program to draw circles at arbitrary centers?

Instead of assuming the center at the origin, two additional variables x_c and y_c are taken as user input. Every calculated point is then translated by adding these center coordinates, allowing the circle to be drawn at any desired location.

13. Conclusion

This experiment successfully implemented the **Midpoint Circle Drawing Algorithm** using OpenGL. The program accepted the radius and center coordinates from the user, generated the circle using the midpoint decision parameter, and displayed it accurately within the specified coordinate system.

The algorithm demonstrated how circle drawing can be performed efficiently using only integer arithmetic and eight-way symmetry. The modifications required by the laboratory, including the **0–100 coordinate system**, **700 × 700 window size**, and **RGB (0–255) color format**, were successfully implemented.

Through this experiment, I gained practical knowledge about raster graphics, pixel-based rendering, OpenGL programming, symmetry in geometric shapes, and the implementation of efficient circle drawing algorithms. The experiment also strengthened my understanding of decision parameters and their importance in computer graphics applications.